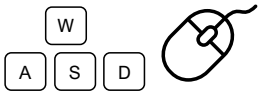
Worksheet 11: First person camera control

Reading	opengl-tutorial. Tutorial 6: Keyboard and Mouse LearnOpenGL: Camera
Purpose 	In this worksheet, we focus on user interaction. Our goal is to give the user the ability to control a camera in a similar manner to games with a first-person perspective. This means using the mouse to rotate the camera (so-called "mouselook") and a set of keys to move the camera around (WASD, for example). To connect this terminology with other types of camera control, we will implement forward and backward motion as dollying and strafe motion as panning of the camera. Dollying is moving along the line of sight. Panning is moving the camera in a plane parallel to the image plane.
Part 1 Mouselook	Pick one of your previous solutions where you draw in 3D and start from there. • Create variables for storing the camera's view direction (the direction from the eye point toward the look-at point) in spherical coordinates. Add a `mousemove` event to the canvas that uses the `movementX` and `movementY` event properties when the primary button is pressed to incrementally update the azimuthal and polar angles of the view direction, respectively. • In each animation frame, compute the look-at point by converting the angles to a Cartesian vector and adding it to the eye position. Use the resulting look-at point to compute the view matrix in the usual way. • Introduce a slider-controlled multiplier for the changes to the angles, giving the user control over mouse sensitivity. • Clamp the polar angle to be within $[0, \pi]$. Note that if you allow the polar angle to hit either 0 or π, the view direction will be colinear with the up direction of the camera, causing the `lookAt` function to fail. You can either handle this as a special case by calculating an alternative up vector (using the azimuthal angle) or you can clamp to a slightly smaller interval than $[0, \pi]$.
Part 2 Quaternion rotation	While the mouselook functionality enables us to look around, it is annoying that the mouse pointer can leave the canvas. Furthermore, the position of the mouse pointer is not particularly relevant to the user, as we only care about its movements. • Add a `click` event to the canvas that switches between locking and unlocking the pointer by calling `canvas.requestPointerLock()` or `document.exitPointerLock()`. Locking the pointer captures it within the canvas (which also makes it invisible). Add a handler for the `pointerlockchange` event that enables/disables control of the camera (check whether the `document.pointerLockElement` is equal to the canvas element to know whether to enable or disable the controls).[1] In the `mousemove` event function, pointer lock or not now replaces the check to see whether the primary button is pressed.

[1] For more info on pointer locking see https://developer.mozilla.org/en-US/docs/Web/API/Pointer_Lock_API

	• **(Optional)** `requestPointerLock` can throw an exception (for example, some browsers do not allow a website to lock the pointer immediately after it is released). Use a try/catch statement to catch this exception and handle it by doing nothing.
Part 3 WASD (dollying and panning)	We would also like to be able to move the eye position. We will use the keyboard for this. • Add four Booleans corresponding to four keys on the keyboard (W, A, S, and D are commonly used). In listeners for the `keydown` and `keyup` events, set/unset the Booleans corresponding to `event.code`. • In each animation frame, compute the time difference since last frame. If the W key (or other key representing forward movement) is pressed, move the eye position along the computed view direction vector by some (optionally slider-controlled) scalar times the time difference. If the S key is pressed, move the eye position along the negated view direction. The difference between the Booleans for the W and S keys can be used. • Take the cross product between the up vector (i.e. the one used in the `lookAt` function) and the view direction vector to obtain a vector pointing to the right of the camera. Based on the state of the A and D keys, move the eye position along this vector. • While moving side-to-side is simple with the current setup, it can be difficult to move up and down. Introduce two more keys (e.g. space and left shift) for moving the camera up and down in world space. Your implementation works as intended if you can use multiple buttons and the mouse simultaneously to navigate the 3D scene.